

```
<style> :root { --accent: #1f4e79; --line: #b7c6d8; --line-strong: #6f87a3; --text: #1f2933; --muted: #52606d; --stripe: #f8fbff; } body { font-family: "Microsoft YaHei", "PingFang SC", sans-serif; color: var(--text); line-height: 1.75; font-size: 14px; } h1, h2, h3, h4 { color: var(--accent); font-weight: 700; } h1 { text-align: center; font-size: 24px; letter-spacing: 0.5px; margin-bottom: 18px; padding-bottom: 10px; border-bottom: 2px solid var(--accent); } h2 { margin-top: 28px; padding-left: 10px; border-left: 5px solid var(--accent); font-size: 20px; } h3 { margin-top: 22px; font-size: 17px; } h4 { margin-top: 18px; font-size: 15px; } p, li { text-align: justify; } blockquote { margin: 14px 0 18px; padding: 10px 14px; color: var(--muted); background: #f7f9fc; border-left: 4px solid var(--accent); } table { width: 100%; margin: 14px 0 20px; border-collapse: collapse; table-layout: fixed; font-size: 13px; } thead { display: table-header-group; } tr { page-break-inside: avoid; } th, td { padding: 9px 10px; border: 1px solid var(--line); vertical-align: top; word-break: break-word; overflow-wrap: anywhere; } th { color: #fff; background: var(--accent); border-color: var(--line-strong); text-align: center; font-weight: 700; } tbody tr:nth-child(even) { background: var(--stripe); } img { display: block; max-width: 92%; margin: 14px auto 18px; } code { padding: 1px 5px; background: #f3f6fa; border-radius: 4px; font-size: 0.95em; } .page-break { page-break-before: always; } </style>
```

智能充电桩调度计费系统_动态结构与静态结构设计

组长：唐振桓
组员1：杨凌俊
组员2：袁炜途
组员3：马迪轩
组员4：刘宇轩
日期：2026/06/02
文档说明：本次作业基于 HW1 已交付的领域模型、用例模型与系统顺序图 (SSD)，进一步细化各系统事件 (除"注册"用例外) 在系统内部的动态协作过程，并补充用例级别的静态结构设计。

目录

- 第一章：系统架构选择及说明 (10%)
 - 1.1 架构选型
 - 1.2 物理部署与运行视图
 - 1.3 三层逻辑分层
 - 1.4 服务器内部职责划分
 - 1.5 与本次作业的对应关系
- 第二章：动态结构设计 (80%)
 - 2.1 设计概览与统一图例
 - 2.2 充电申请用例的系统事件
 - 2.3 查看账单 / 查看详单用例的系统事件
 - 2.4 运行充电桩用例的系统事件
 - 2.5 充电桩状态查询及队列状态查询用例的系统事件
 - 2.6 调度对象的核心交互 (组长完成)
 - 2.7 可选加分调度策略
- 第三章：静态结构设计 (10%)
 - 3.1 用例级类图
 - 3.2 类属性与方法说明表

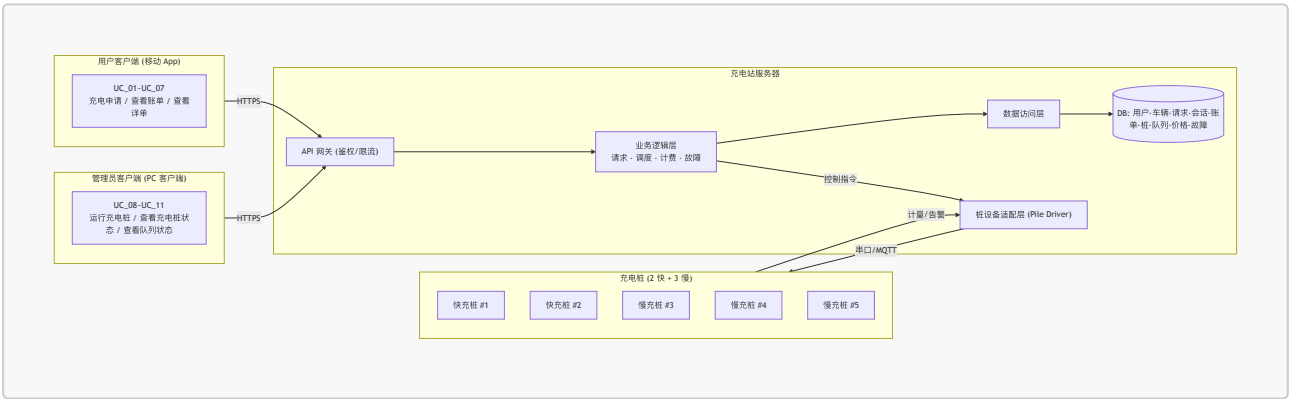
第一章：系统架构选择及说明 (10%)

1.1 架构选型

综合需求说明书的三个核心约束——(1) 系统由"服务器端 + 用户客户端 + 管理员客户端"三方构成、(2) 排队调度、计费 and 故障恢复均需要全局状态一致、(3) 充电桩状态需要在客户端定时刷新——本系统选择"客户端-服务器架构 (C/S) + 三层逻辑分层 + 服务端 MVC"的复合方案，并在服务器内部按职责进一步划分为"边界 (Boundary) — 控制 (Control) — 实体 (Entity)"三类协作对象，便于在第二章用动态图清晰地表达每一个系统事件。

放弃 B/S 纯浏览器方案的原因：管理员客户端需要长连接接收充电桩状态轮询和故障告警，用户客户端需要长连接接收"叫号"推送，而本系统又部署在校内私网，C/S 在长连接、即时通知和本地缓存方面优于一般 B/S。放弃微服务方案的原因：5 个充电桩 + 单一充电站的部署规模不构成分服务的必要业务边界，分布式事务反而会显著增加调度一致性维护成本。

1.2 物理部署与运行视图



充电桩设备本身不直接对接客户端：所有控制指令（启动、停止、参数下发）和状态回采（计量、告警）都经过服务器的"桩设备适配层"，这样可以保证状态在服务器端有唯一可信副本，便于调度。

1.3 三层逻辑分层

层	主要职责	对应类型示例
表示层 (Presentation)	屏幕渲染、表单校验、推送接收、本地缓存	UserClient / AdminClient 及其内部 View/ViewModel
业务逻辑层 (Business Logic)	用例级控制逻辑、调度算法、计费规则、超时与故障策略	ChargeRequestController / ScheduleManager / BillingService / FaultService / PileService
数据访问层 (Data Access)	持久化、事务、唯一性约束、并发控制	RequestRepo / SessionRepo / BillRepo / PileRepo / PriceRepo

1.4 服务器内部职责划分

服务器在业务逻辑层内部进一步采用 BCE (Boundary-Control-Entity) 模式：

- **Boundary (边界对象)**：每个用例对应一个 XxxController，承担鉴权、参数校验、把客户端协议消息映射成内部消息。
- **Control (控制对象)**：跨多个实体协调的服务对象，如 SchedulerManager (排队/分桩)、BillingService (计费)、FaultService (故障)。
- **Entity (实体对象)**：HW1 领域模型中 18 个类的实现，承担属性持久化和不变量自检。

第二章每张时序图中的 lifeline 都遵循该划分，命名方式为 :ClassName 表示对象，斜杠/箭头采用 UML 2 标准记法。

1.5 与本次作业的对应关系

- 本章 (10%) 给出整体架构图与分层划分，作为后续两章的协作上下文。
- 第二章 (80%) 中所有系统事件均落在"用户/管理员客户端 → API 网关 → Controller → Service → Entity → Repo"这条链路上；调度策略与故障恢复增加 SchedulerManager 和 FaultService 之间的内部交互。
- 第三章 (10%) 仅保留与系统事件直接相关的类，仅使用依赖、定向关联与继承三种关系，符合作业要求。

第二章：动态结构设计 (80%)

2.1 设计概览与统一图例

本章共给出 15 个必做系统事件的时序图、3 个组长必做的调度对象交互图，以及 2 个可选加分调度策略的时序图，合计 20 张 UML Sequence Diagram。

为保证图与图之间的一致性，所有时序图遵循以下约定：

1. 全部使用 UML 2 sequence diagram 记法；同步消息使用实心三角箭头，异步消息使用细线开放箭头，返回值使用虚线箭头。
2. lifeline 命名遵循 角色 / :类名 的形式；同一个类在不同时序图中保持同一命名。
3. 控制结构使用 alt / opt / loop / par 复合片段，约束信息写在 note 中。
4. 系统事件名 (来自作业要求表格) 在第一条消息上以加粗形式显示，参数列表与原表格完全一致。
5. 与 HW1 操作契约保持一致：前置条件不满足则进入 alt 异常分支返回错误码 1；正常路径返回 0 或题目要求的对象。

为便于参与对象交叉引用，下表列出贯穿全章的核心对象：

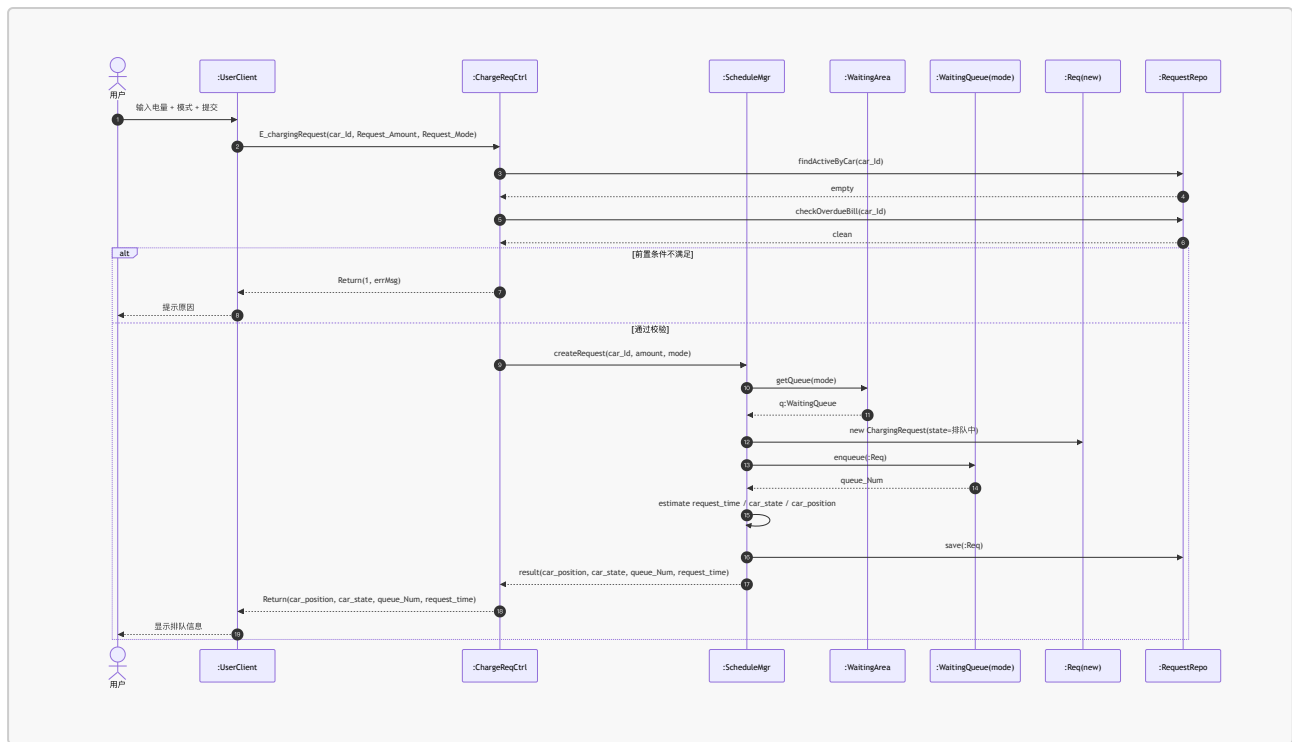
简写	全名	所在层	角色
:UserClient	用户客户端	表示层	用户操作入口
:AdminClient	管理员客户端	表示层	管理员操作入口
:ChargeReqCtrl	:ChargeRequestController	业务边界	充电申请系列用例的 Controller
:BillCtrl	:BillController	业务边界	账单/详单用例的 Controller

简写	全名	所在层	角色
:PileCtrl	:PileController	业务边界	运行/查询充电桩用例的 Controller
:QueueCtrl	:QueueController	业务边界	查看队列状态用例的 Controller
:ScheduleMgr	:ScheduleManager	业务控制	调度算法与队列调度入口
:BillingSvc	:BillingService	业务控制	计费、生成账单与详单
:FaultSvc	:FaultService	业务控制	故障识别、再调度、故障恢复
:PileSvc	:PileService	业务控制	充电桩生命周期 (上电/启动/关闭)
:WaitingArea	等候区	实体	持有两个等待队列
:WaitingQueue	等待队列 (快/慢)	实体	FIFO 队列
:ChargingArea	充电区	实体	持有 5 个充电桩
:ChargingPile	充电桩	实体	桩内 4 位排队队列
:Req	:ChargingRequest	实体	单次请求生命周期对象
:Session	:ChargingSession	实体	充电会话 (实时电量、时长)
:Bill	:Bill	实体	一次会话对应一份账单/详单
:PricingRule	分时电价规则	实体	三段式电价
:ServiceRule	服务费规则	实体	服务费率
:PileDriver	桩设备适配层	设备	与物理桩交换控制/计量信号

命名映射说明：作业要求表中的英文消息名 (例如 `E_chargingRequest`) 是"系统事件"层面，对应 HW1 操作契约中的 `SubmitChargeRequest` 等编程名。本章在第一条消息上严格使用作业要求表中的命名，以满足"标黄系统事件"的覆盖要求。

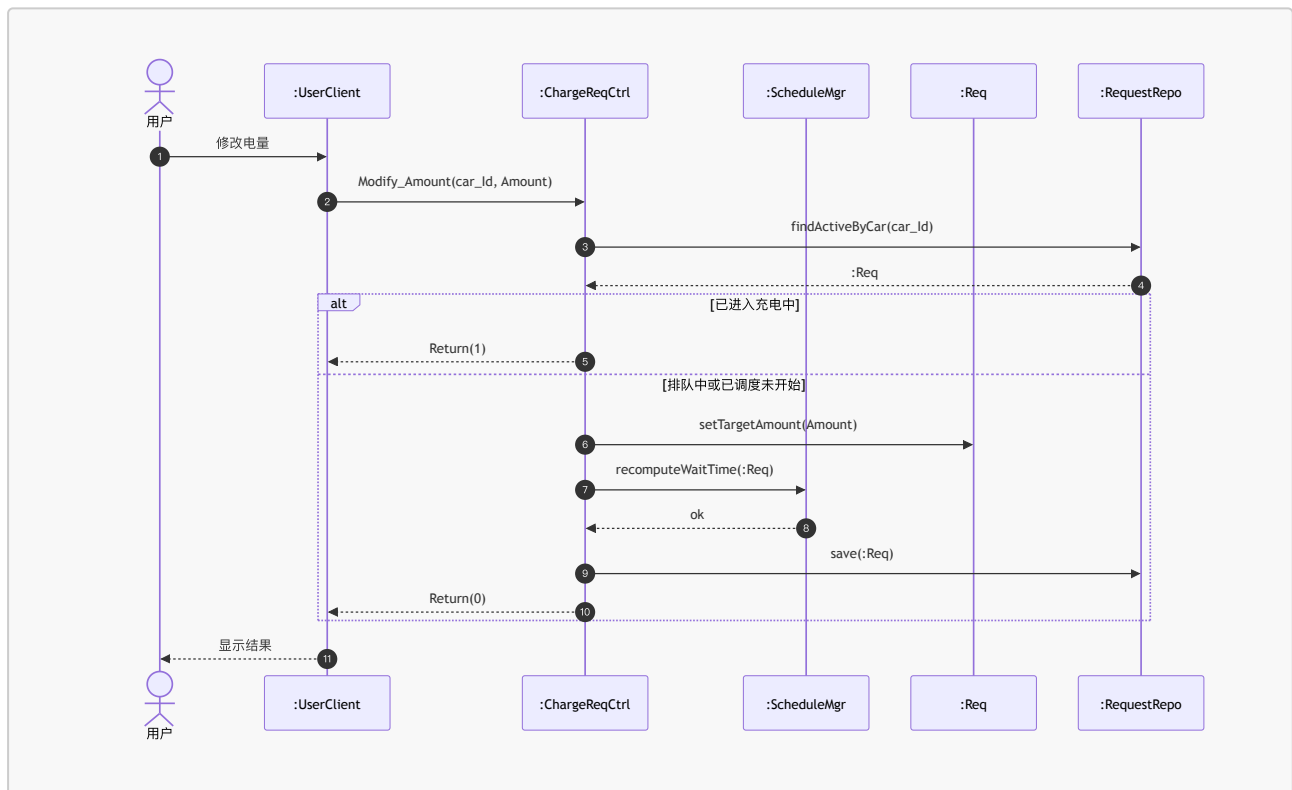
2.2 充电申请用例的系统事件

2.2.1 提交充电申请 `E_chargingRequest(car_Id, Request_Amount, Request_Mode)`

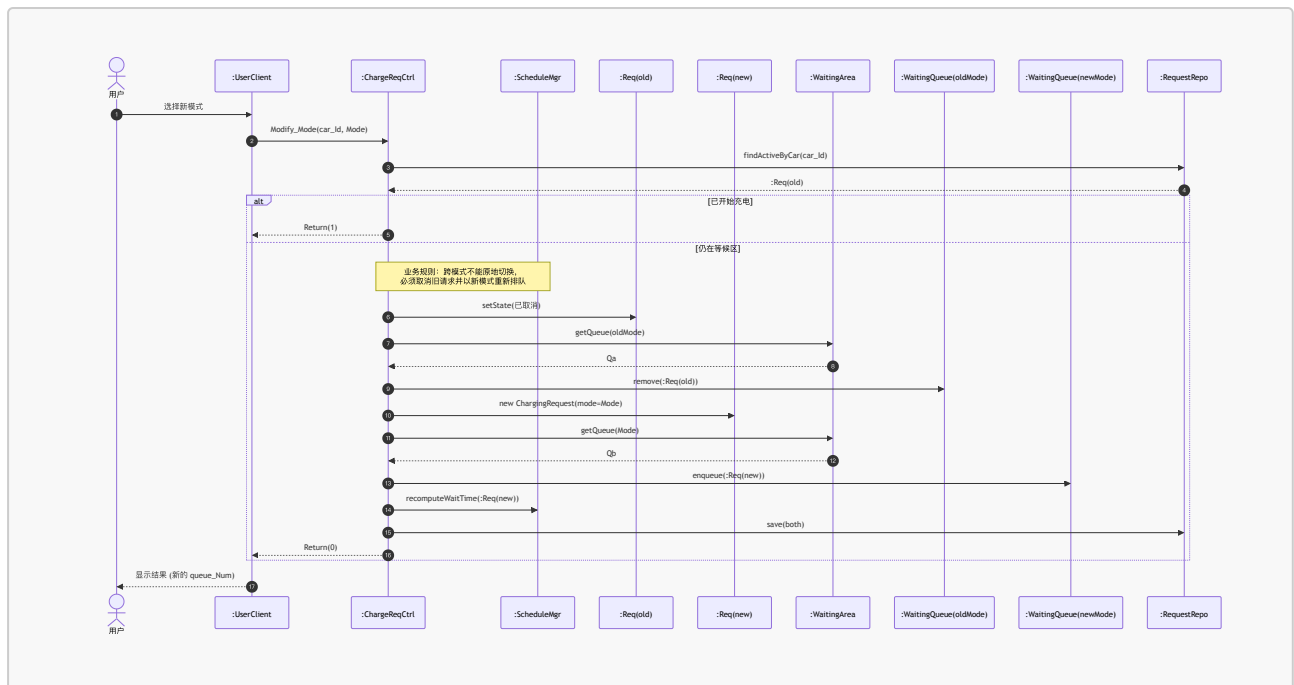


说明：`car_state` 取值与 HW1 一致，含"排队中 / 已调度未开始 / 充电中 / 已完成 / 已取消"。

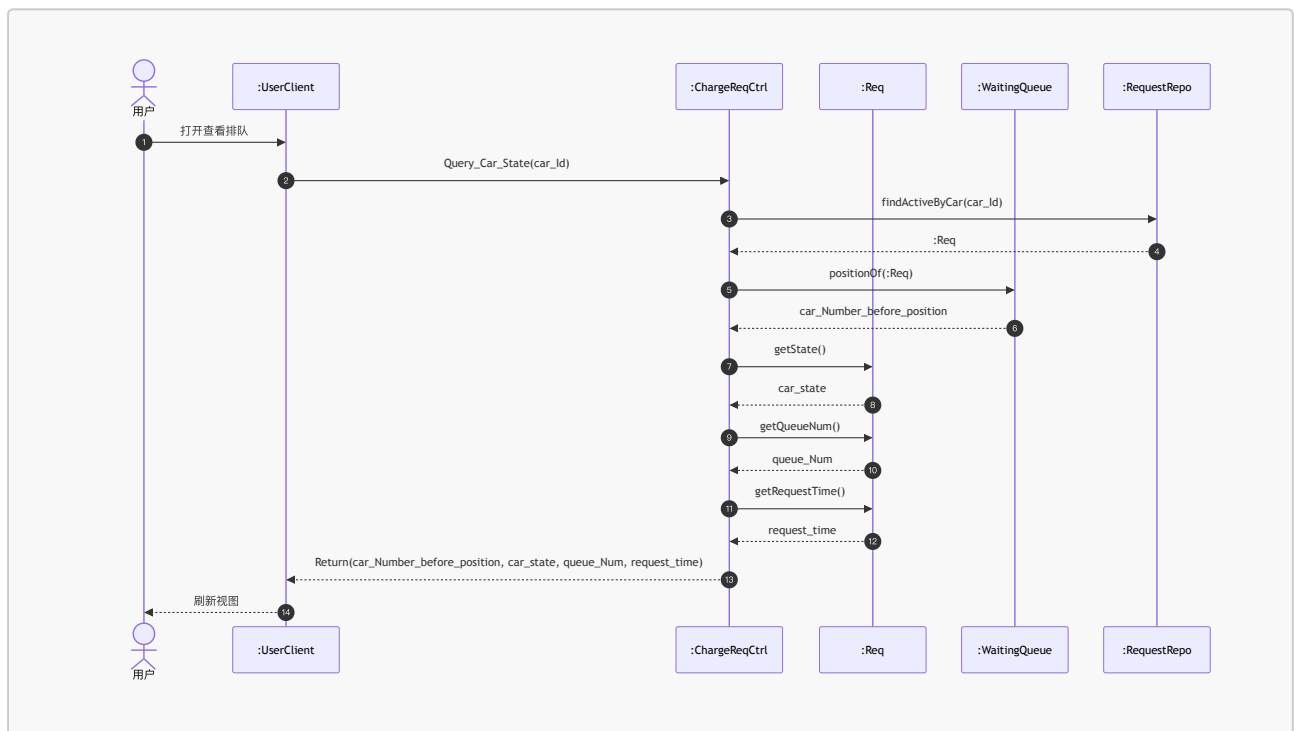
2.2.2 修改充电量 `Modify_Amount(car_Id, Amount)`



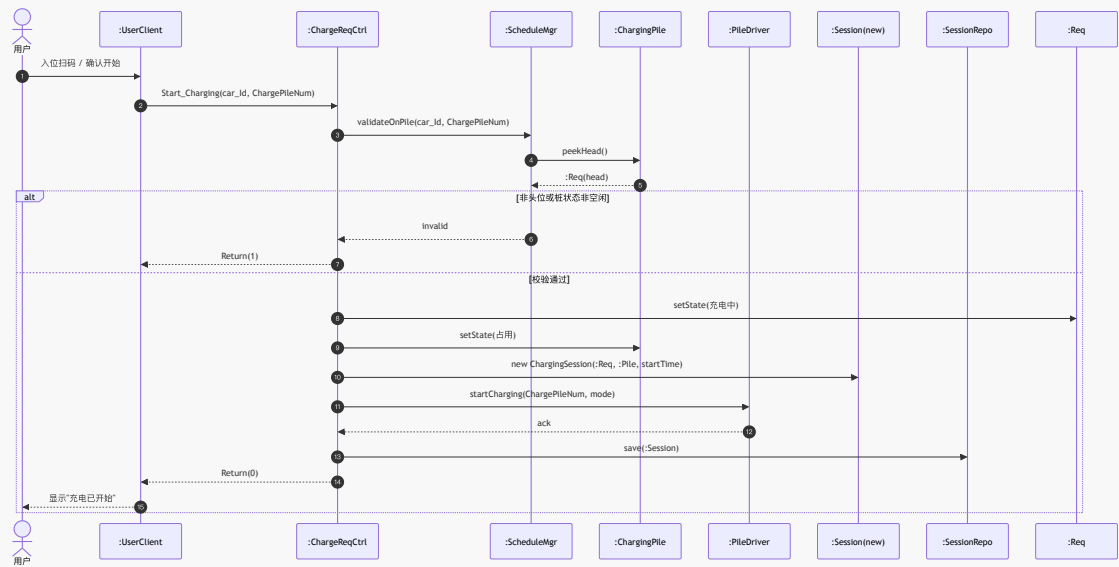
2.2.3 修改充电模式 `Modify_Mode(car_Id, Mode)`



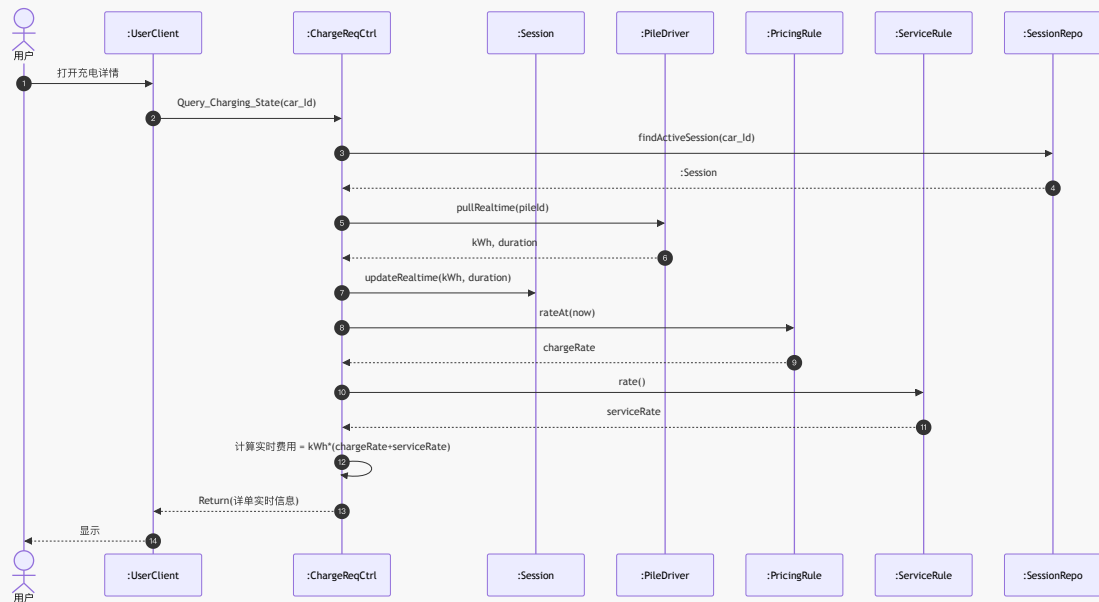
2.2.4 查看队列状态 Query_Car_State(car_Id)



2.2.5 开始充电 Start_Charging(car_Id, ChargePileNum)

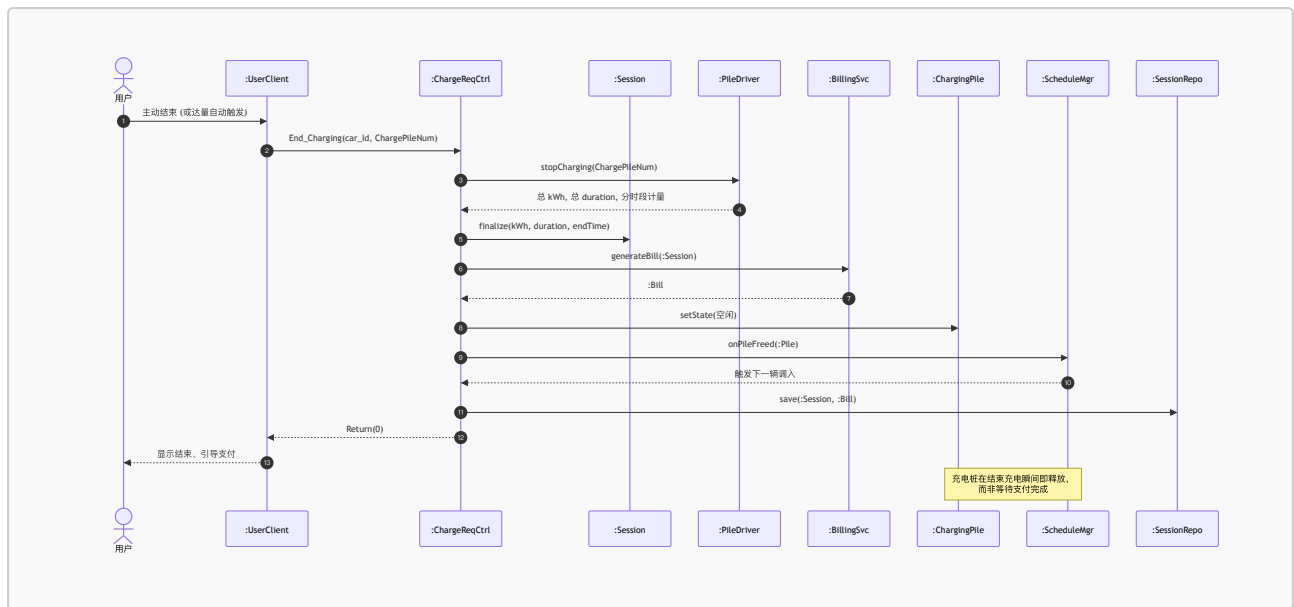


2.2.6 查看充电状态 Query_Charging_State(car_Id)



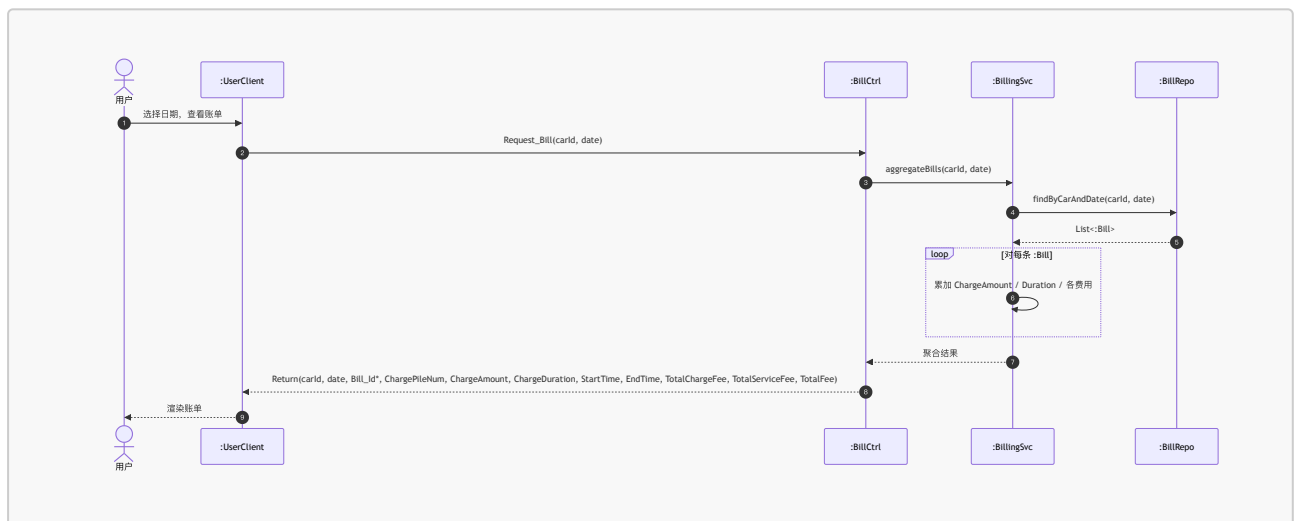
返回字段按需求说明含：pileNum, startTime, 当前 kWh, 当前 duration, 估算 chargeFee, 估算 serviceFee, 当前总费用，与"详单信息"字段一一对应。

2.2.7 结束充电 End_Charging(car_Id, ChargePileNum)

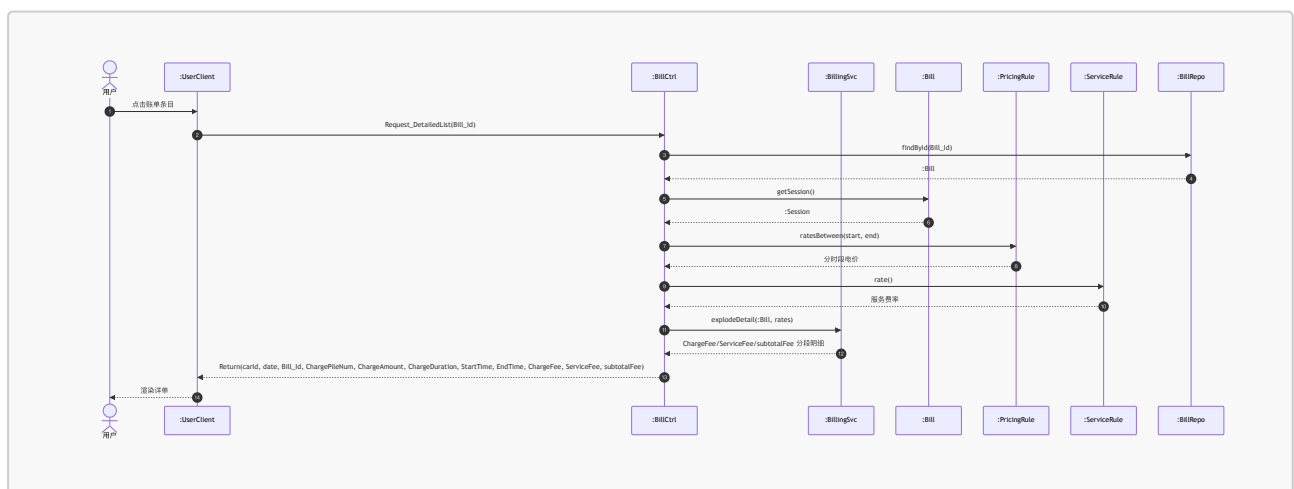


2.3 查看账单 / 查看详单用例的系统事件

2.3.1 查看账单申请 `Request_Bill(carId, date)`

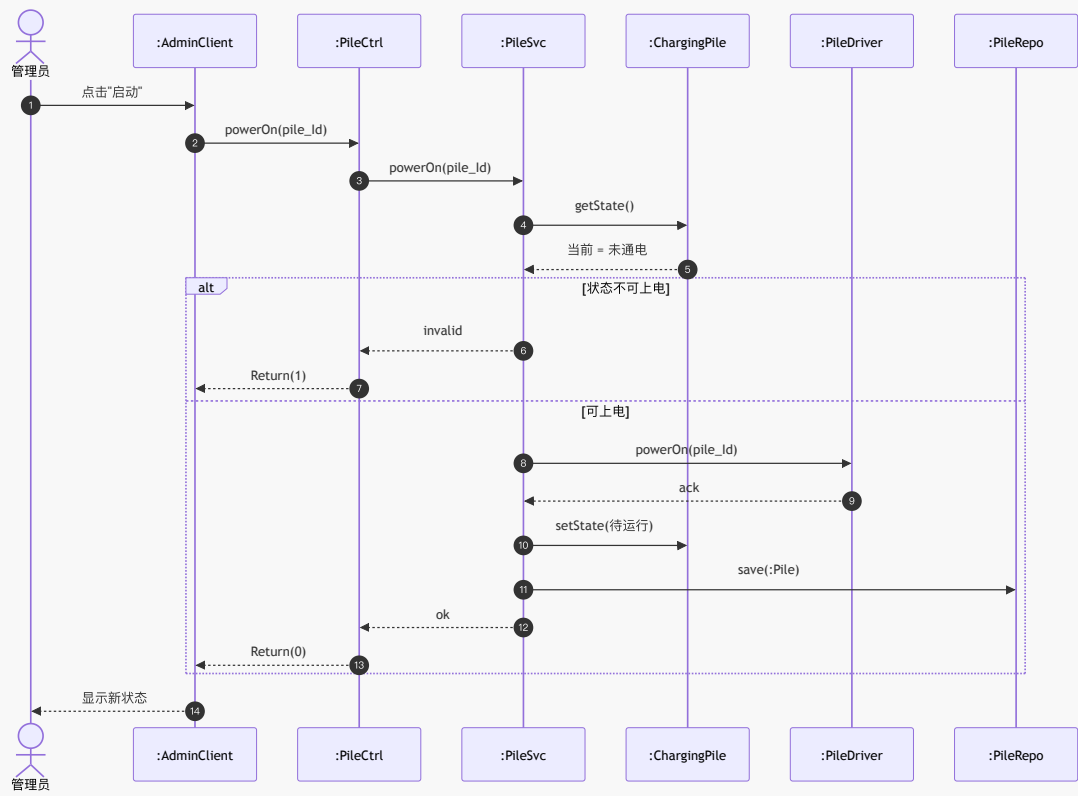


2.3.2 查看详单申请 `Request_DetailedList(Bill_Id)`

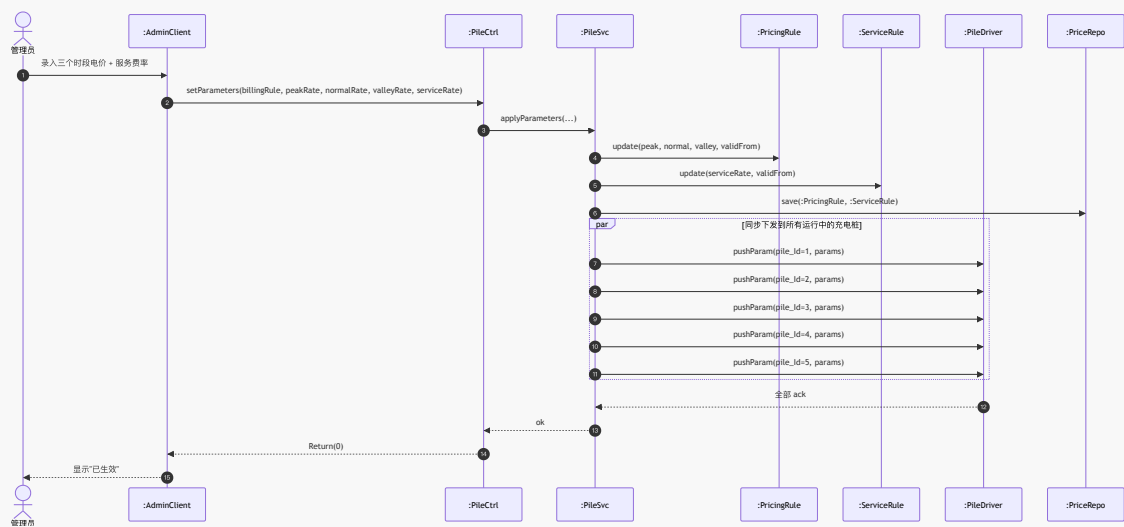


2.4 运行充电桩用例的系统事件

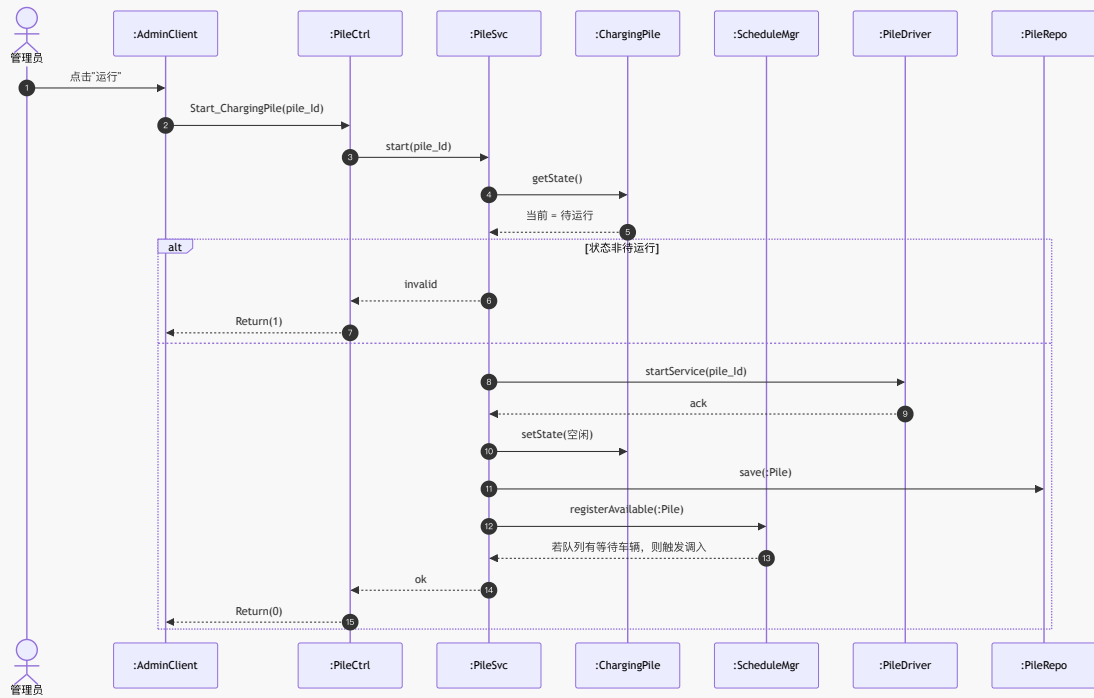
2.4.1 启动充电桩 `powerOn(pile_Id)`



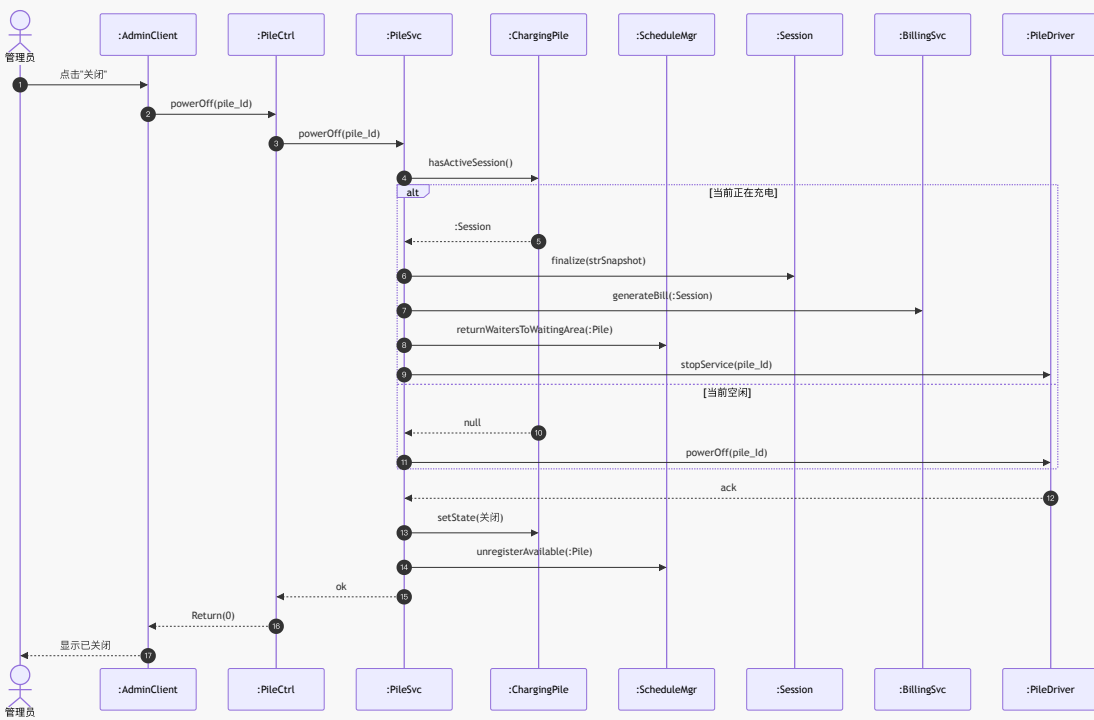
2.4.2 设置参数 setParameters(计费规则，三个时段的电价数据等)



2.4.3 运行充电桩 Start_ChargingPile(pile_Id)

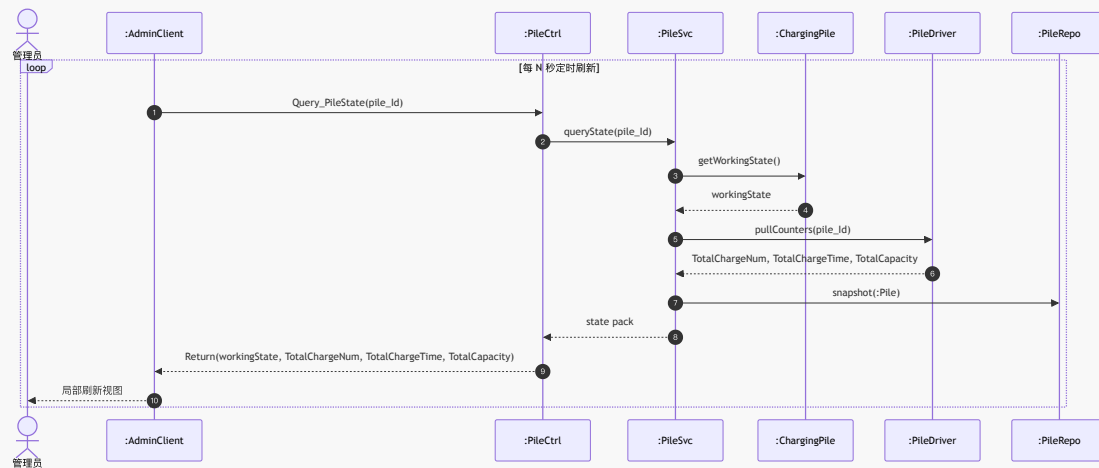


2.4.4 关闭充电桩 powerOff(pile_Id)



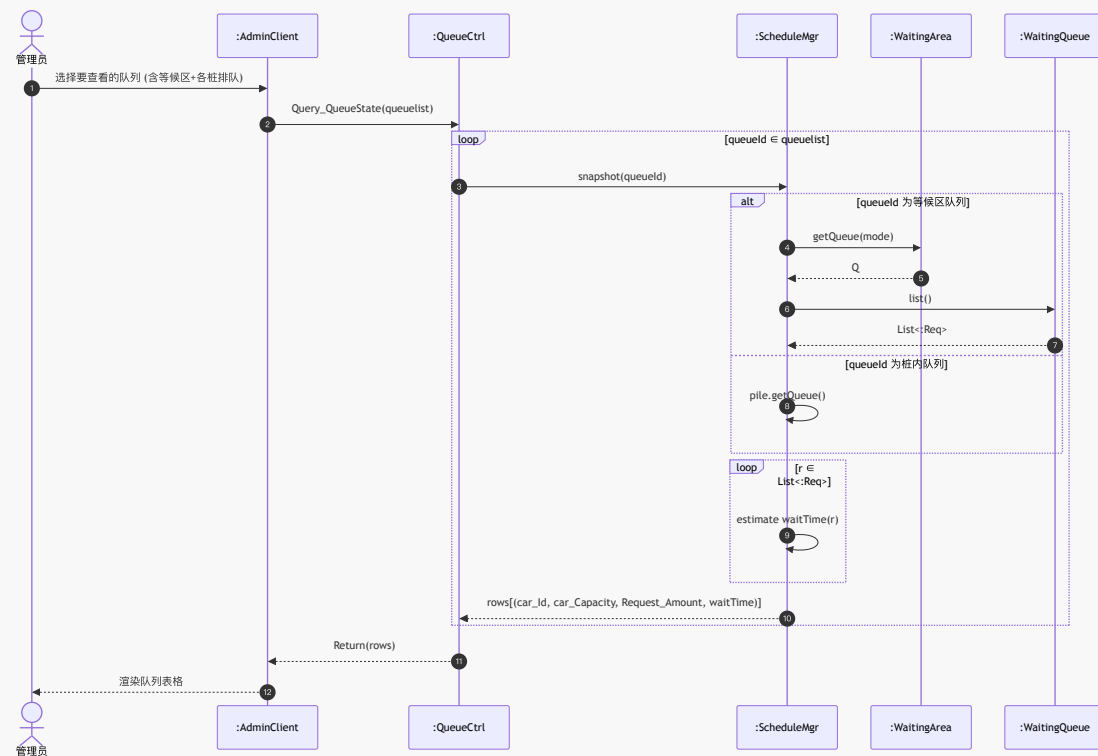
2.5 充电桩状态查询及队列状态查询用例的系统事件

2.5.1 查看充电桩状态 Query_PileState(pile_Id) (定时刷新)



客户端依据 N (默认 5s) 周期发起请求；当 pile_Id 为空时由 :PileCtrl 内部展开为 5 个并行 queryState。

2.5.2 查看队列状态 Query_QueueState(queueList)

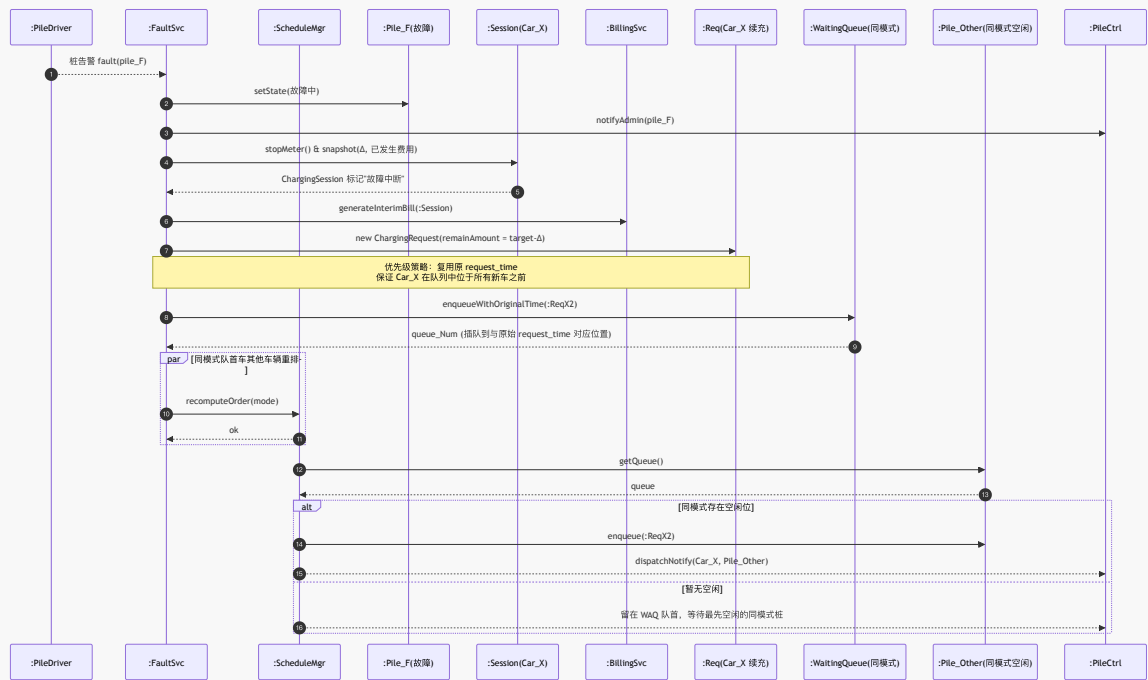


2.6 调度对象的核心交互 (组长完成)

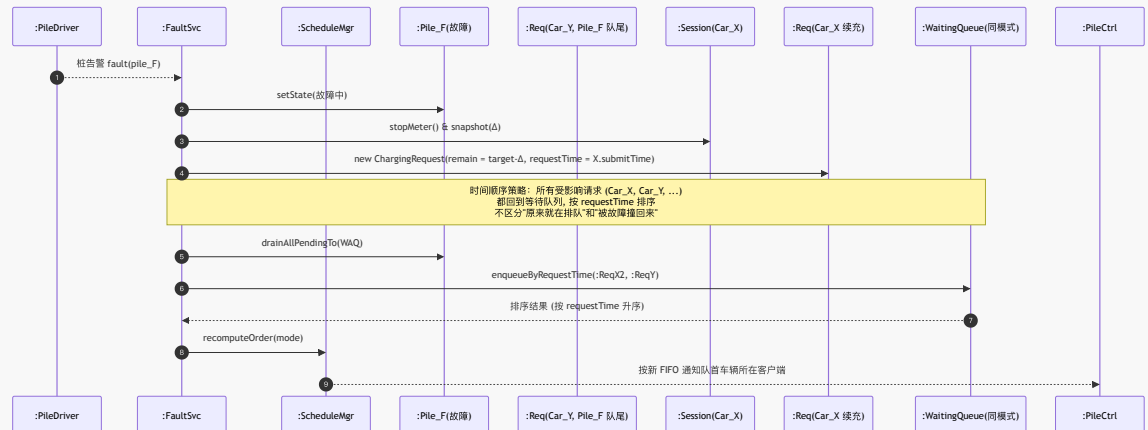
本节由组长 唐振桓 完成，针对充电过程中出现一个充电桩故障的情况，分别给出 (1) 优先级调度、(2) 时间顺序调度、(3) 充电中故障恢复 三种调度对象之间的交互过程。

公共前提：5 个充电桩中 Pile_F 在某辆车 Car_X 充电过程中发生硬件故障。Car_X 已充入 Δ 度电、Pile_F 队尾还排有 Car_Y (尚未开始充电)。下文三张图分别使用不同的"再调度"策略来处理。

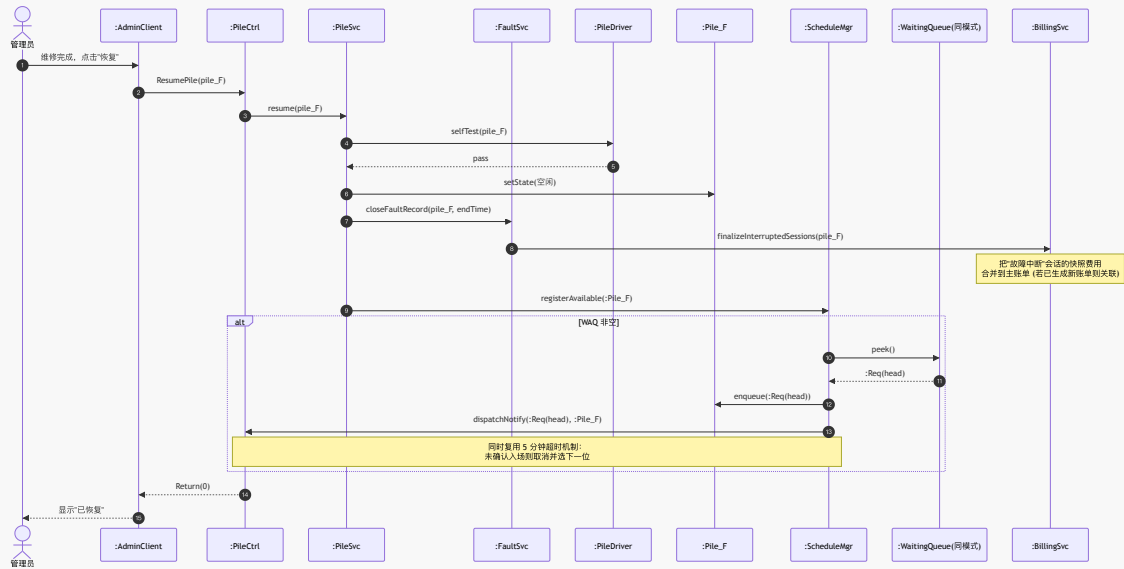
2.6.1 优先级调度 (故障车辆保留原优先级，绕过同模式新车直接被同模式空闲桩接走)



2.6.2 时间顺序调度 (故障车辆按其原始提交时间回到等待队列, 与排队中的车辆按 FIFO 重排)



2.6.3 充电中故障恢复 (维修完成后, **Pile_F** 重新参与同模式调度)

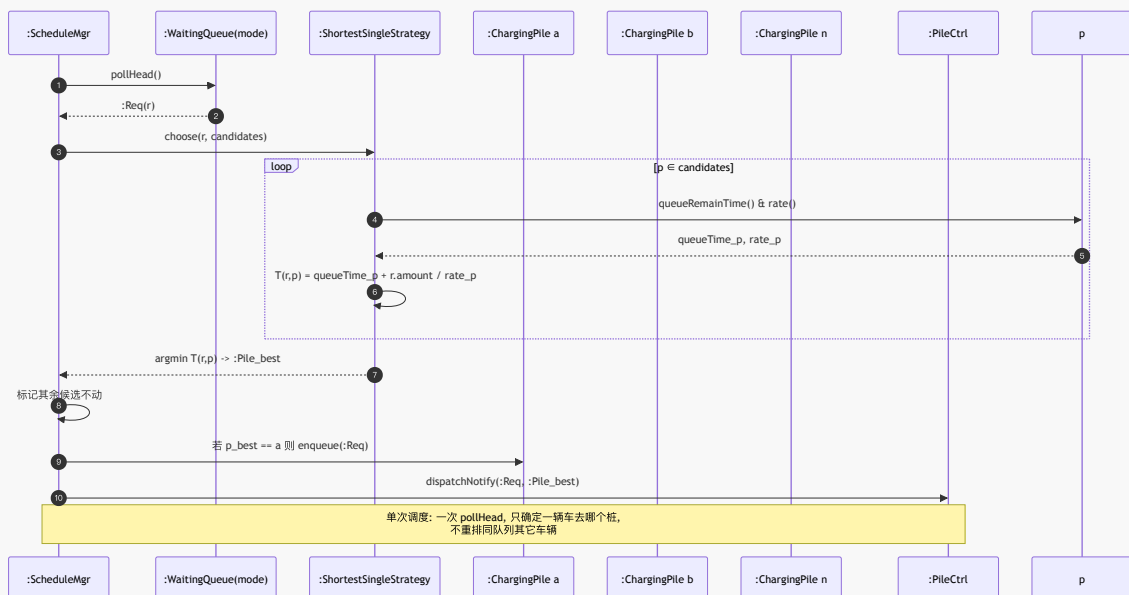


2.7 可选加分调度策略

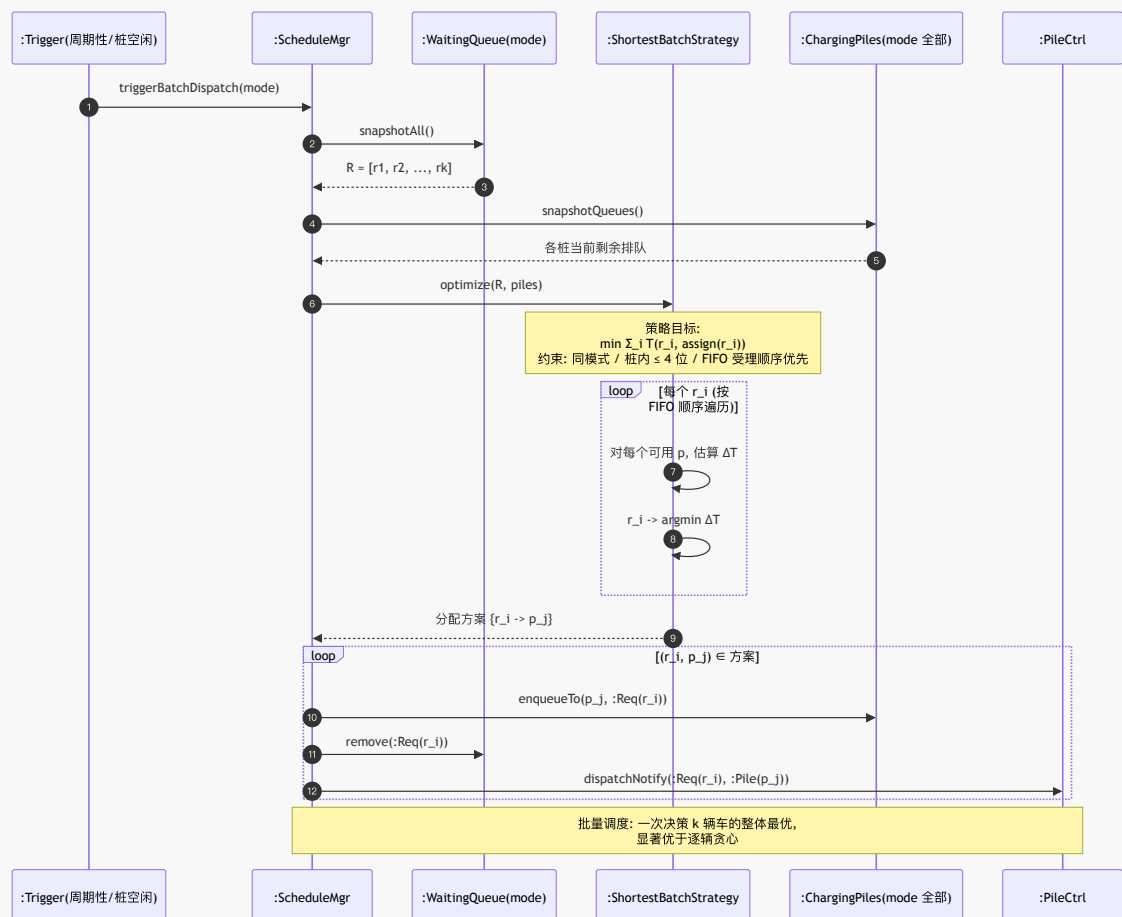
本节面向作业要求中的两项可选加分项：单次调度总充电时长最短、批量调度总充电时长最短。两项均聚焦"完成充电所需时间 = 等待时长 + 自己充电时长"的最小化策略，且不与 2.6.1/2.6.2 冲突，而是作为"被调度对象进入充电区瞬间"的桩选择策略。

形式化定义：设当前模式空闲桩集合为 P^* ，对每个候选 $p \in P^*$ ，候选请求 r 选择 p 的总完成时间为 $T(r, p) = \text{sum}(\text{remain_charge_time of } p\text{'s current queue}) + r.\text{amount} / p.\text{rate}$ 。

2.7.1 单次调度总充电时长最短



2.7.2 批量调度总充电时长最短

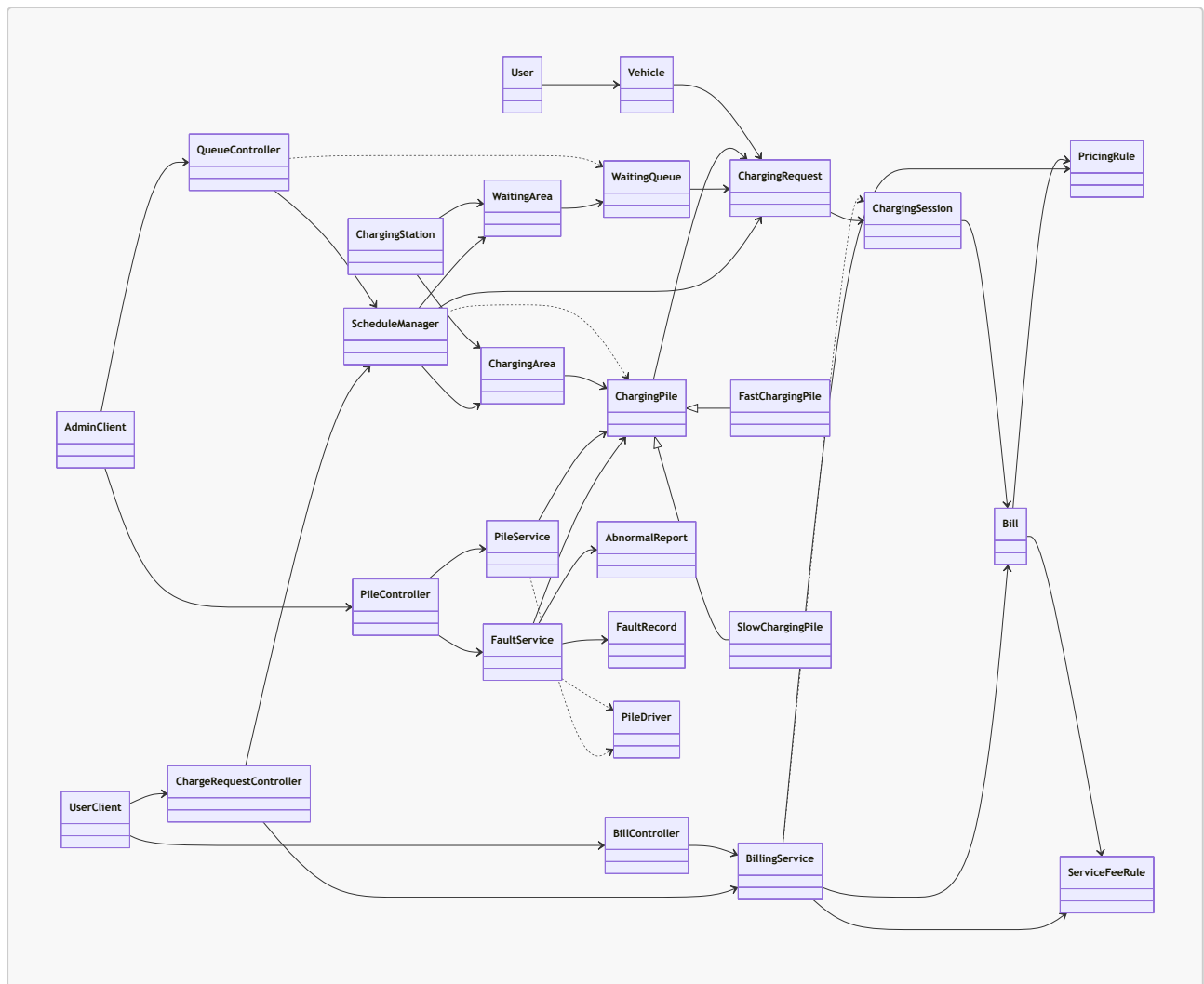


第三章：静态结构设计 (10%)

3.1 用例级类图

下图覆盖第二章全部系统事件涉及的"边界 / 控制 / 实体"对象，按作业要求只使用三种关系：

- **继承 (Generalization)**：▷，仅用于充电桩家族。
- **定向关联 (Directed Association)**：➡，表示一个类持有另一个类的引用。
- **依赖 (Dependency)**：-.->▷，表示一个类在方法签名/调用中使用另一个类，但不持有其引用。



备注：图中刻意避免使用聚合 / 组合 / 关联类等 HW1 已使用过的关系符号；本次只保留作业要求的三种关系，所有"整体-局部"语义统一弱化为"定向关联"。

3.2 类属性与方法说明表

由于类图中各类只标了类名，下面按"边界 / 控制 / 实体 / 设备"四组给出属性与方法。仅列与本次系统事件直接相关的成员；继承自父类的成员不在子类中重复列出。

(1) 边界类 (Controller)

类名	主要属性	主要方法 (与系统事件对应)
UserClient	currentUserId, currentVehicleId, sessionToken	submitChargeRequest(), modifyAmount(), modifyMode(), queryCarState(), startCharging(), queryChargingState(), endCharging(), requestBill(), requestDetailedList()
AdminClient	adminId, refreshInterval	powerOn(), setParameters(), startChargingPile(), powerOff(), queryPileState(), queryQueueState(), resumePile()
ChargeRequestController	—	E_chargingRequest, Modify_Amount, Modify_Mode, Query_Car_State, Start_Charging,

类名	主要属性	主要方法 (与系统事件对应)
		Query_Charging_State, End_Charging
BillController	—	Request_Bill, Request_DetailedList
PileController	—	powerOn, setParameters, Start_ChargingPile, powerOff, Query_PileState, ResumePile, notifyAdmin
QueueController	—	Query_QueueState

(2) 控制类 (Service / Manager)

类名	主要属性	主要方法
ScheduleManager	strategy: 调度策略, waitingArea, chargingArea	createRequest(), recomputeWaitTime(), validateOnPile(), onPileFreed(), registerAvailable(), unregisterAvailable(), snapshot(), triggerBatchDispatch()
BillingService	pricingRule, serviceRule	generateBill(), generateInterimBill(), aggregateBills(), explodeDetail(), finalizeInterruptedSessions()
FaultService	activeFaults: Map<pileId, FaultRecord>	onPileFault(), drainAllPendingTo(), closeFaultRecord(), recomputeOrderAfterFault()
PileService	—	powerOn(), powerOff(), start(), applyParameters(), queryState(), resume()

(3) 实体类 (Entity)

类名	主要属性	主要方法
ChargingStation	stationId, name, address	getWaitingArea(), getChargingArea()
WaitingArea	capacity	getQueue(mode)
ChargingArea	spotPerPile=4	listPiles(), countFreeSpots()
WaitingQueue	mode: 快/慢, items: List	enqueue(), enqueueWithOriginalTime(), enqueueByRequestTime(), pollHead(), peek(), positionOf(), list(), remove(), snapshotAll()

类名	主要属性	主要方法
ChargingPile	pileId, type, state, totalChargeNum, totalChargeTime, totalCapacity, rate, queue: List	getState(), setState(), enqueue(), peekHead(), hasActiveSession(), getQueue(), queueRemainTime(), pullCounters()
FastChargingPile	rate=30kW (默认)	— (继承 ChargingPile)
SlowChargingPile	rate=7kW (默认)	— (继承 ChargingPile)
User	userId, userName, blockedByOverdue	hasOverdueBill(), addVehicle()
Vehicle	vehicleId, carCapacity	getOwner(), getActiveRequest()
ChargingRequest	requestId, carId, mode, targetAmount, state, requestTime, queueNum, originalRequestTime	setState(), setTargetAmount(), getState(), getQueueNum(), getRequestTime()
ChargingSession	sessionId, requestId, pileId, startTime, endTime, kWh, duration, interruptFlag, snapshotFee	updateRealtime(), finalize(), stopMeter(), snapshot()
Bill	billId, sessionId, chargeAmount, chargeDuration, startTime, endTime, totalChargeFee, totalServiceFee, totalFee, payState	computeFromSession(), getDetail(), markPaid()
PricingRule	peakRate, normalRate, valleyRate, peakRange, valleyRange, validFrom	rateAt(time), ratesBetween(t1,t2), update()
ServiceFeeRule	rate, validFrom	rate(), update()
AbnormalReport	reportId, userId, pileId, description, reportTime	bindTo(pile)
FaultRecord	faultId, pileId, faultType, faultTime, endTime, sourceReportId	open(), close()

(4) 设备类 (Driver)

类名	主要属性	主要方法
PileDriver	endpoints: Map<pileId, ConnectionInfo>	powerOn(), powerOff(), startCharging(), stopCharging(), startService(), stopService(), pushParam(), pullRealtime(), pullCounters(), selfTest()

第四章：系统事件人员分配

说明：本节按作业要求给出"系统事件 → 责任人"的分工，便于评审追踪。组长部分由唐振桓承担；其余系统事件均匀分给四位组员。统稿、文档美化、交叉检查仍由全组协同。

4.1 系统事件分工表

类别	系统事件	主要负责人	协作
充电申请	E_chargingRequest	杨凌俊	唐振桓
充电申请	Modify_Amount	杨凌俊	袁炜途
充电申请	Modify_Mode	袁炜途	杨凌俊
充电申请	Query_Car_State	刘宇轩	杨凌俊
充电申请	Start_Charging	马迪轩	唐振桓
充电申请	Query_Charging_State	马迪轩	袁炜途
充电申请	End_Charging	袁炜途	唐振桓
查看账单	Request_Bill	刘宇轩	袁炜途
查看详单	Request_DetailedList	刘宇轩	袁炜途
运行充电桩	powerOn	马迪轩	杨凌俊
运行充电桩	setParameters	马迪轩	袁炜途
运行充电桩	Start_ChargingPile	杨凌俊	唐振桓
运行充电桩	powerOff	杨凌俊	马迪轩
查看充电桩状态	Query_PileState	刘宇轩	马迪轩
查看队列状态	Query_QueueState	袁炜途	刘宇轩
调度交互 (组长)	优先级调度	唐振桓	全组
调度交互 (组长)	时间顺序调度	唐振桓	全组
调度交互 (组长)	充电中故障恢复	唐振桓	全组
加分项	单次调度总充电时长最短	唐振桓 + 袁炜途	杨凌俊
加分项	批量调度总充电时长最短	唐振桓 + 杨凌俊	刘宇轩

4.2 文档与统稿分工

工作项	唐振桓	杨凌俊	袁炜途	马迪轩	刘宇轩
第一章 系统架构选择及说明	√	√			
第二章 时序图绘制 (Mermaid)		√	√	√	√
第三章 用例级类图与说明表	√		√		√

工作项	唐振桓	杨凌俊	袁炜途	马迪轩	刘宇轩
调度对象交互 (2.6 节)	√				
加分项策略 (2.7 节)	√	√	√		
Markdown 排版与目录				√	√
全文复核与提交	√				√

评分说明：本节按作业第 4 条要求列出系统事件的人员分配。若漏写本节，按作业要求最高扣 5 分；本节本身不计入正分。